External Validation of a Positive-Closure Proof Architecture

A Multi-Tranche Survey: Twelve Benchmark Dimensions in Lean

(portability of the comparison pattern, not a universality theorem)

Nova Spivack

2026

Abstract

Transferability paper. This manuscript establishes that the positive-closure architecture is *exportable* to ordinary mathematics outside the flagship Infinity Compression modules; it is not the program map and not the venue for theorem-level leverage in algebra, topology, or arithmetic—those roles belong to the companion general-method papers cited below. The Infinity Compression (IC) formalization develops a layered positive-closure proof architecture: collapse to a bare certificate, a typed forgetful map, fiber structure over that certificate, and, where appropriate, canonical witnesses or sections. A natural methodological concern is that this architecture may be endogenous to the flagship NEMS-style case rather than a reusable proof pattern. This paper presents a **consolidated machine-checked external validation**: twelve independent tranches of ordinary mathematics, each formalized in Lean without importing the IC spine, each comparing a *flat baseline* (the obvious Mathlib-shaped existence or representability statement) with an *architecture-guided* theorem bundle (nonempty fibers, surjectivity of the forgetful map, `HasRightInverse`, an identified `RightInverse` section, and a uniform canonical witness in each fiber), *except* tranche **T12**, which records an explicit **MODERATE** / control pattern for a *non-surjective* collapse.
Tranche **T1** is the setoid quotient benchmark (collapse by an equivalence relation; baseline `Quotient.exists_rep`). Tranche **T2** is the Cartesian product with first projection (many enriched pairs over one bare image—*multi-route* without quotienting). Tranche **T3** is the dependent sum $\Sigma_{b:B} E\, b$ with `Sigma.fst` (bundle projection; dependent generalization of T2). Tranches **T4**–**T7** cover sum discriminants, subtype projection, orbit quotients, and ideal quotients. Tranches **T8**–**T11** add pushout glueing, set membership as classifier, localization via fraction pairs, and pullbacks (sheaf-style fiber products). Tranche **T12** is `Nat.succ`: fibers are nonempty exactly off zero, with no global right inverse. The contribution is not a catalog of new classical theorems; it is a demonstration that the same architectural lens organizes *twelve* external families and yields strengthened, reusable proof interfaces where the rubric applies. Under the project's validation rubric, tranches T1–T11 support a **STRONG** verdict; T12 documents a complementary **MODERATE** boundary case. Companion general-method papers then show that the same architecture becomes load-bearing in theorem-level settings, including group extension splitting criteria, embedding problems from inverse Galois theory, and Quillen's Theorem A for Galois connections. We situate the work in the NEMS / APS / IC / Strata stack and state explicitly that APS is not tested by these tranches.
Trust boundary: This is a twelve-tranche portability survey in Lean; it is not a universality theorem over all mathematical settings and does not subsume APS abstract-indexed separation results.

Keywords: formal verification, positive closure, fibers, external validation, Mathlib, Lean 4, Infinity Compression architecture

MSC 2020: 03B70, 18A30, 68V15

Code/Data availability: https://github.com/novaspivack/infinity-compression-lean; external-validation tranches under `InfinityCompression/` (no import of the flagship `IC` spine per tranche design); `lake build`; see `lakefile.lean` and repository `MANIFEST.md`.

1 Premises, disagreement coordinates, and trust boundary

Where disagreement can (and cannot) live

Formal spine. Tranche verdicts (**STRONG** / **MODERATE**) reference machine-checked bundles in the stated Lean modules, not informal analogy alone.

Premise coordinates (for external readers).

- **Transfer vs. flagship.** Tranches deliberately avoid importing the internal flagship spine; conclusions concern portability of a *pattern*, not identity with NEMS paper claims. *Logical effect:* Rejecting one tranche does not automatically refute the flagship internal-origins paper.
- **Rubric, not omnicoverage.** Twelve families evidence export of an architectural lens; the paper explicitly disclaims a single theorem quantifying over all mathematics. *Logical effect:* Negative search outside the tranche list is independent of this validation's internal logic.

Contents

1 Premises, disagreement coordinates, and trust boundary **2**

2 Introduction **4**
2.1 Executive interpretation . . . 4

3 The positive-closure architecture as a reusable schema **4**

4 Unified methodology: baseline vs. architecture-guided bundle **5**

5 Tranche T1: Setoid quotients **5**
5.1 Why quotients . . . 5
5.2 Lean formalization and main bundle . . . 5
5.3 What the architectural packaging adds . . . 5

6 Tranche T2: Cartesian product and first projection **6**
6.1 Mathematical story . . . 6
6.2 Baseline vs. bundle . . . 6

7 Tranche T3: Dependent sums and the bundle projection **6**
7.1 Mathematical story . . . 6
7.2 Hypothesis and classical section . . . 7

8 Tranche T4: Sum types and discriminant collapse **7**

9 Tranche T5: Subtype projection **7**

10 Tranche T6: Orbit quotient from a group action **7**

11 Tranche T7: Ideal quotient **8**

12 Tranche T8: Pushout / colimit glueing **8**

13 Tranche T9: Membership classifier **8**

14 Tranche T10: Localization pairs **9**

15 Tranche T11: Pullback / fiber product **9**

16 Tranche T12: Non-surjective collapse (MODERATE) **9**

17 Comparative synthesis across tranches **9**
17.1 What this validation shows (and does not) 11

18 Unified validation verdict and the meaning of STRONG **11**

19 Place in the larger program **11**

20 Beyond retrospective validation: group extensions **12**
20.1 Faithfully flat descent 12

21 Limitations and future work **13**

A Theorem indices (Lean names by module) **13**

B A workflow for reusing this pattern **17**

2 Introduction

The Infinity Compression (IC) program develops a positive-closure proof architecture organized around four layers: a collapsed bare layer, a projection from enriched to bare structure, fibers over bare points, and, when available, canonical witnesses or sections. In the flagship IC formalization this architecture is load-bearing: it organizes the distinction between theorem-level certification and richer reflective realization. **The central claim of this paper is that the positive-closure architecture is a reusable proof pattern—not merely a custom narrative for the flagship case—and that claim is supported by multiple independent external benchmarks.**

A single quotient formalization already begins to answer the worry that the architecture might be idiosyncratic; this document *consolidates* that quotient tranche with eleven additional standard constructions—products, dependent sums, coproduct discriminants, subtypes, orbit and ideal quotients, pushouts, classifiers, localization pairs, pullbacks, and a non-surjective control—so that the evidence base is explicitly multi-family. The result is one self-contained validation paper: readers need not cross-reference a separate "survey" manuscript for the product and bundle stories.

Scope and place in the series. We do not claim a universal theorem that every closure problem in mathematics must be formalized in IC's types. We claim that, in Lean, on twelve concrete external families, the architecture exports cleanly, yields structured theorem bundles beyond flat baselines, and aligns with the flagship forgetful / fiber / section narrative [1]. This paper establishes *transferability* (external validity of the packaging). Companion general-method papers [7, 8] supply the theorem-level mathematical leverage: group extension splitting criteria, embedding problems from inverse Galois theory, and Quillen's Theorem A for Galois connections. A program-level map tying internal origin, transfer, and discharges together is [9]; the distilled summit claim is [10]. The principal value of the tranche survey is methodological transfer; it must not be read as a substitute for those theorem papers or for the synthesis manuscript.

2.1 Executive interpretation

The twelve tranches are not twelve unrelated curiosities: together they show that the same *packaging*—bare layer, forgetful map, fibers, optional sections—survives across quotient-like collapses (T1, T6–T8), projection- and bundle-like fibers (T2, T3, T5, T9, T11), a structured algebraic presentation layer (T10), and a non-surjective boundary (T12). That is evidence for *portability* of the architecture across ordinary Mathlib-shaped mathematics; it is *not* a single theorem quantifying over all mathematical settings, and it does not by itself supply the theorem-level depth of the group-extension or Quillen manuscripts—which is exactly why those papers exist.

Suite context. The Infinity Compression paper suite is organized as follows: the *flagship manuscript* states the *internal origin theorem*; the *transferability paper* validates the same packaging on twelve external benchmarks; *general-method manuscripts* (group extensions; Quillen) supply theorem-level leverage as the *algebraic and arithmetic engine* and *topology discharge*; the *synthesis manuscript* maps the full program; the *summit manuscript* distills reflective non-exhaustion; the *technical Mathlib companion* is reviewer-facing. This manuscript is the *transferability paper*.

3 The positive-closure architecture as a reusable schema

Let π denote a forgetful projection from an *enriched* carrier to a *bare* certificate space (a collapsed layer where theorem-level records live). Write E for an enriched carrier and B for a bare layer. The

schema is:

$$E \xrightarrow{\pi} B, \qquad \mathrm{Fib}(b) := \pi^{-1}(b) \ \text{ for } b \in B, \qquad \text{optionally } s : B \to E \text{ with } \pi \circ s = \mathrm{id}_B. \tag{1}$$

Here $\mathrm{Fib}(b)$ is the fiber over a bare point; a section s is a global choice of preimage.

Each tranche below instantiates (1) with different E, B, and π. The quotient tranche uses Mathlib's quotient map and `Quotient.out`; the product tranche uses first projection and a parametric section $x \mapsto (x, y_0)$; the bundle tranche uses `Sigma.fst` and a `noncomputable` section from classical choice when $\forall b,\ \mathrm{Nonempty}(E\,b)$.

4 Unified methodology: baseline vs. architecture-guided bundle

In every tranche we record:

- a **baseline** statement profile matching how Mathlib users typically see the mathematics (existence of representatives, trivial points in fibers, etc.);
- an **architecture-guided bundle**: named bare layer, named projection, fiber type as a subtype or structured copy, surjectivity of π, `HasRightInverse`, a concrete `RightInverse` witness, and a `canonical...Witness` in each fiber.

The methodological gain is *interface structure*: the bundle is a reusable template parallel to the flagship story, not a claim of new classical content.

5 Tranche T1: Setoid quotients

5.1 Why quotients

This tranche is a positive-closure benchmark in the sense relevant to IC: it exhibits collapse, fibers, and a uniform section-like recovery mechanism (`Quotient.out`), not only an existential shadow.

The quotient construction is standard mathematics (see, e.g., [6]); it has a genuine collapse map from carriers to quotient points; it admits a classical baseline (`Quotient.exists_rep`); and it supports a section story via `Quotient.out`. The benchmark imports only `Mathlib/Data/Setoid/Basic.lean` and `Mathlib/Logic/Function/Basic.lean`.

5.2 Lean formalization and main bundle

All definitions and theorems live in `InfinityCompression/Validation/QuotientFiberBenchmark.lean` (namespace `InfinityCompression.Validation`). Layering: `BareQuotient`, `EnrichedCarrier`, `forgetToQuotient`, `QuotientFiber`, `canonicalQuotientFiberWitness`.

5.3 What the architectural packaging adds

Classically, every quotient class has *some* representative (`Quotient.exists_rep`). The architecture-guided view reorganizes the same fact under (1): surjectivity, `HasRightInverse`, identification of `Quotient.out` as `RightInverse`, and `canonicalQuotientFiberWitness`. Methodologically, this upgrades a pointwise existential into a named, reusable interface—the same move the positive-closure architecture is designed to expose internally.

Lean name	Meaning	Baseline vs. architecture
`quotient_fiber_nonempty`	each quotient point has a witness in its `QuotientFiber`	baseline-plus (typed fiber nonempty)
`forgetToQuotient_surjective`	`forgetToQuotient` hits every bare point	architecture
`forgetToQuotient_hasRightInverse`	projection admits a global right inverse	architecture
`quotientOut_rightInverse_forgetToQuotient`	`Quotient.out` is a `RightInverse` of `forgetToQuotient`	architecture
`canonicalQuotientFiberWitness`	uniform canonical witness in each fiber	architecture

Table 1: T1 (quotient): main named results in the architecture-guided bundle.

6 Tranche T2: Cartesian product and first projection

6.1 Mathematical story

Let X and Y be types. The first projection $\pi : X \times Y \to X$ forgets the second coordinate. For fixed nonempty Y, many pairs (x, y_1) and (x, y_2) share the same bare image x: this is *multi-route over one bare certificate* without any quotient—a clean stress test for "fiber + section" language distinct from equivalence collapsing.

6.2 Baseline vs. bundle

Baseline: once Y is inhabited, each $x : X$ has some pair mapping to it (e.g. (x, y_0) for a chosen y_0). **Architecture bundle:** we package `ProductFiberAt X Y x`, surjectivity of `forgetFirst`, `HasRightInverse`, the parametric section `productSection y0`, `RightInverse` compatibility, and `canonicalProductFiberWitness`. The Lean module is `ProductProjectionFiberBenchmark.lean`; it imports only `Mathlib.Logic.Function.Basic`.

Lean name	One-line reading
`forgetFirst`	first projection $X \times Y \to X$.
`ProductFiberAtXYx`	fiber of `forgetFirst` over `x` (`Y` explicit in the name).
`product_fiber_nonempty`	each bare `x` has a nonempty fiber (given `y0:Y`).
`forgetFirst_surjective`	`forgetFirst` is surjective (given `y0`).
`forgetFirst_hasRightInverse`	surjectivity ⇒ `HasRightInverse`.
`productSection_rightInverse_forgetFirst`	`productSectiony0` is `RightInverse` to `forgetFirst`.
`canonicalProductFiberWitness`	distinguished point `â§Í(x,y0),rflâ§Ï` in each fiber.

Table 2: T2 (product projection): architecture-guided bundle.

7 Tranche T3: Dependent sums and the bundle projection

7.1 Mathematical story

For $E : B \to \text{Type}$, the dependent sum $\Sigma_{b:B} E\, b$ is the standard *bundle* total space; `Sigma.fst` projects to the base. The fiber over $b : B$ is (equivalent to) $E\, b$. This generalizes T2: product is the special case of a constant family $E\, b = Y$.

7.2 Hypothesis and classical section

We assume $\forall b : B,\ \mathrm{Nonempty}(E\,b)$ so that `Sigma.fst` is surjective. The global section `sigmaSection` uses `Classical.choose` per fiber; the module is `noncomputable` where required. Baseline: each base point has *some* fiber point. Bundle: `SigmaFiber`, surjectivity, `HasRightInverse`, `sigmaSection_rightInverse_sigma_fst`, `canonicalSigmaFiberWitness`.

Lean name	One-line reading
`SigmaFiberEb`	points in $\Sigma\,E$ lying over `b`.
`sigma_fiber_nonempty`	nonempty fibers from $\forall b,\ \mathrm{Nonempty}(E\,b)$.
`sigma_fst_surjective`	`Sigma.fst` surjective.
`sigma_fst_hasRightInverse`	`HasRightInverse` for `Sigma.fst`.
`chooseFiber` / `sigmaSection`	classical section of the bundle.
`sigmaSection_rightInverse_sigma_fst`	`RightInverse` law.
`canonicalSigmaFiberWitness`	uniform witness in each `SigmaFiber`.

Table 3: T3 (dependent sum): architecture-guided bundle (`SigmaFiberBenchmark.lean`).

8 Tranche T4: Sum types and discriminant collapse

The coproduct `Sum A B` carries a natural *tag-forgetting* map to `Bool` (`forgetSumIsLeft`), parallel in spirit to T2's projection but for coproducts rather than products: many `inl` values share the same bare tag. The module is `SumCoproductFiberBenchmark.lean`. Baseline: both sides inhabited to hit both booleans. Bundle: `SumFiberAt`, surjectivity, `sumSection`, `RightInverse` law, `canonicalSumFiberWitness`.

Lean name	One-line reading
`forgetSumIsLeft`	collapse `Sum A B` to a `Bool` side tag.
`SumFiberAtABb`	fiber over a tag `b : Bool`.
`sumSection`	section `Bool → Sum A B` at fixed witnesses.
`forgetSumIsLeft_surjective`	surjectivity (given `a0`, `b0`).
`sumSection_rightInverse_forgetSumIsLeft`	`RightInverse` law.
`canonicalSumFiberWitness`	canonical witness in each `SumFiberAt`.

Table 4: T4 (sum / coproduct): architecture-guided bundle.

9 Tranche T5: Subtype projection

For a predicate `p` on a type α, `Subtype.val` forgets proofs. Under the hypothesis $\forall a,\ p\ a$, `Subtype.val` is surjective onto α; fibers are singletons. Module: `SubtypeValFiberBenchmark.lean`.

10 Tranche T6: Orbit quotient from a group action

Given `MulAction G X`, the orbit relation `same_orbit` yields a `Setoid` and quotient `BareOrbitQuotient`; the collapse map is `to_orbit_quotient`. Module: `OrbitRelationQuotientFiberBenchmark.lean`.

Lean name	One-line reading
`SubtypeFiberAt`	fiber of `Subtype.val` over `a : α`.
`subtype_val_surjective`	surjectivity from $\forall a,\ p\ a$.
`subtypeSection`	section sending `a` to `<a, h a>`.
`subtypeSection_rightInverse_subtypeVal`	`RightInverse` law.
`canonicalSubtypeFiberWitness`	canonical witness in each fiber.

Table 5: T5 (subtype): architecture-guided bundle.

Lean name	One-line reading
`same_orbit`	orbit relation $\exists g,\ g \cdot x = y$.
`grp_action_orbit_setoid`	`Setoid` for the orbit relation.
`to_orbit_quotient`	`Quotient.mk` from `X` to orbit space.
`to_orbit_quotient_surjective`	surjectivity.
`quotient_out_rightInverse_to_orbit_quotient`	`Quotient.out` as `RightInverse`.
`canonical_orbit_quotient_fiber_witness`	canonical witness in each fiber.

Table 6: T6 (orbit quotient): architecture-guided bundle.

11 Tranche T7: Ideal quotient

For a commutative ring `R` and ideal `I`, the map `Ideal.Quotient.mk I` collapses to the quotient R/I. This is the standard algebraic quotient parallel to T1's setoid quotient. Module: `IdealQuotientFiberBenchmark.lean`.

Lean name	One-line reading
`forgetToIdealQuotient`	`Ideal.Quotient.mk I`.
`IdealQuotientFiber`	fiber over a bare coset in R/I.
`forgetToIdealQuotient_surjective`	surjectivity (via `Quotient.exists_rep`).
`idealQuotientOut_rightInverse_forgetToIdealQuotient`	`Quotient.out` as `RightInverse`.
`canonicalIdealQuotientFiberWitness`	canonical witness in each fiber.

Table 7: T7 (ideal quotient): architecture-guided bundle.

12 Tranche T8: Pushout / colimit glueing

Glueing `inl a0` with `inr b0` in `Sum A B` is presented as the equivalence generated by `Relation.EqvGen` on explicit symmetric edges (`PushoutCoequalizerFiberBenchmark.lean`).

13 Tranche T9: Membership classifier

Subsets as maps to `Prop` and `Subtype` of membership in `S`: `SetClassifierFiberBenchmark.lean` packages membership fibers and global sections when every `x` lies in `S`.

Lean name	One-line reading
pushoutRel	generating edges for glueing.
forgetToPushout	Quotient.mk to the pushout type.
forgetToPushout_surjective	surjectivity.
quotientOut_rightInverse_forgetToPushout	Quotient.out as RightInverse.
canonicalPushoutFiberWitness	canonical witness in each fiber.

Table 8: T8 (pushout): architecture-guided bundle.

Lean name	One-line reading
membershipClassifier	characteristic predicate x in S.
ClassifierFiber	fiber over x for Subtype.val.
classifier_fiber_nonempty_of_mem	nonempty fiber when x lies in S.
subtypeVal_surjectiveOn	surjectivity of Subtype.val under global membership.
classifierSection_rightInverse_subtypeVal	RightInverse for the section.

Table 9: T9 (classifier / subset): architecture-guided bundle.

14 Tranche T10: Localization pairs

Fraction presentation mk' from R × M is surjective; LocalizationPairFiberBenchmark.lean uses IsLocalization.sec as canonical preimage data.

Lean name	One-line reading
forgetPairToLocalization	IsLocalization.mk' on pairs.
forgetPairToLocalization_surjective	pair-map surjectivity.
sec_rightInverse_forgetPairToLocalization	IsLocalization.sec as RightInverse.
canonicalLocalizationPairFiberWitness	canonical pair in each fiber.

Table 10: T10 (localization pairs): architecture-guided bundle.

15 Tranche T11: Pullback / fiber product

Given f : A → C, g : B → C, the subtype of A × B with f a = g b is the pullback; pullbackFst projects to A (PullbackFiberBenchmark.lean).

16 Tranche T12: Non-surjective collapse (MODERATE)

Nat.succ is not surjective; fibers over n > 0 are nonempty singletons. NonSurjectiveSuccFiberBenchmark.lean records ¬HasRightInverse succ and pred_leftInverse_succ.

17 Comparative synthesis across tranches

Conceptual grouping.

Lean name	One-line reading
`PullbackType`	fiber-product subtype.
`pullbackFst`	first projection (renamed to avoid product-tranche clash).
`pullbackFst_surjective`	surjectivity assuming overlap (for all `a` some `b` matches).
`pullbackSection_rightInverse_pullbackFst`	chosen section is `RightInverse`.

Table 11: T11 (pullback): architecture-guided bundle.

Lean name	One-line reading
`succFiber`	fiber of `succ` over `n`.
`succFiber_nonempty_iff`	nonempty iff `n` is not zero.
`nat_succ_not_surjective`	obstruction to global section.
`pred_leftInverse_succ`	standard left-inverse identity.

Table 12: T12 (MODERATE control): `Nat.succ`.

Tr.	Bare B	Projection π	What varies in the fiber	Section flavor
T1	quotient	`forgetToQuotient` / `Quotient.mk`	equivalence classes	`Quotient.out` (canonical)
T2	X	`forgetFirst`	second coordinate $y : Y$	`productSection` y0 (parametric)
T3	B	`Sigma.fst`	dependent $E\,b$	`sigmaSection` (`noncomputable`)
T4	`Bool`	`forgetSumIsLeft`	tagged `Sum` values	`sumSection` (parametric)
T5	α	`Subtype.val`	proof data (singleton fibers)	`subtypeSection`
T6	orbit quotient	`to_orbit_quotient`	orbit classes	`Quotient.out` (canonical)
T7	R/I	`forgetToIdealQuotient`	cosets mod I	`Quotient.out` (canonical)
T8	pushout	`forgetToPushout`	`EqvGen` classes	`Quotient.out` (canonical)
T9	α	`Subtype.val` on `S`	membership fibers	`classifierSection`
T10	localized ring	`forgetPairToLocalization`	denominator pairs	`IsLocalization.sec`
T11	A	`pullbackFst`	compatible B-legs	`pullbackSection` (`noncomputable`)
T12	`Nat`	`Nat.succ`	predecessor fibers	`Nat.pred` (left inverse only)

Table 13: Side-by-side view: twelve instantiations (T12 is a MODERATE boundary case).

- **Quotient-like collapse:** T1, T6, T7, T8 (equivalence classes, orbits, ideals, pushout glueing).
- **Projection / bundle fibers:** T2, T3, T5, T9, T11 (products, dependent sums, subtypes, classifiers, pullbacks).
- **Structured algebraic presentation:** T10 (localization pairs).
- **Non-surjective boundary:** T12 (`Nat.succ`—local structure without global section).

17.1 What this validation shows (and does not)

Validates: that the positive-closure comparison pattern is portable across the twelve families under the stated rubric.

Does not validate: a universal metatheorem; automatic theorem-level depth in algebra or topology. The *general-method manuscripts* [7, 8] exist precisely for that latter kind of leverage.

18 Unified validation verdict and the meaning of STRONG

Under the project's validation rubric [3], a benchmark is rated **STRONG** when it exports the architecture cleanly to external mathematics and yields a theorem bundle structurally richer than the obvious baseline. Tranches **T1**–**T11** are recorded with that ambition; **T12** is an explicit **MODERATE** / methodological control: non-surjective collapse with nonempty fibers only on the image. *Collectively*, the tranches strengthen the claim that the pattern is not tied to a single family of examples.

STRONG encodes methodological transfer and corollary structure, not difficulty of raw mathematical content. No tranche proves a new "summit" theorem for the IC landscape; together they support external validity of the *packaging*.

19 Place in the larger program

- **IC flagship (internal):** the positive-closure architecture and main certification / reflective splits—load-bearing inside the library [1].
- **External validation (this paper):** machine-checked demonstrations that the same pattern applies *outside* the flagship modules, on quotients, products, dependent sums, coproduct discriminants, subtypes, orbit and ideal quotients, pushouts, classifiers, localization pairs, pullbacks, and a non-surjective control tranche.
- **General-method theorem papers [7, 8]:** the same fiber/section/obstruction architecture applied to theorem-level mathematics—group extension splitting, embedding problems, Quillen's Theorem A for Galois connections. These papers demonstrate that the architecture is not merely organizational but mathematically load-bearing.
- **Synthesis and summit:** the cross-domain program map [9] and the reflective non-exhaustion summit statement [10] (not to be confused with this transferability survey).
- **Strata:** abstract synthesis and bridge laws [2].
- **APS:** local-to-global backbone elsewhere in the stack. **These tranches do not test APS directly;** they test reusable positive-closure *packaging* on external mathematics.
- **NEMS:** outer burden and interface story for the paper suite.

20 Beyond retrospective validation: group extensions

The twelve tranches above apply the architecture to problems whose flat baselines were already provable. A stronger test is whether the decomposition helps on problems where the distinction between layers is *load-bearing*—where the bare and enriched levels carry genuinely different mathematical content.

Group extensions provide exactly this test. A short exact sequence $1 \to N \to E \xrightarrow{\pi} G \to 1$ always admits a set-theoretic section (the bare layer: π is surjective, so `HasRightInverse` holds). But a *homomorphic* section—a splitting—is additional algebraic structure that may or may not exist. The obstruction is a 2-cocycle valued in N: given any set-theoretic section σ, define $c(g_1, g_2) = \sigma(g_1)\,\sigma(g_2)\,\sigma(g_1 g_2)^{-1}$, which lands in the kernel $\mathrm{im}(\iota) \cong N$. The extension splits if and only if some section has identically trivial cocycle.

We formalize this in three Lean modules: `GeneralMethod/GroupExtension/FiberArchitecture.lean`, `GeneralMethod/GroupExtension/SchurZassenhaus.lean`, and `GeneralMethod/GroupExtension/LocalGlobal.lean`, building on Mathlib's `GroupExtension` API. The main results, all with zero `sorry`:

- **Fiber architecture.** `ExtensionFiber S g` is the preimage $\pi^{-1}(g)$; fibers are nonempty (surjectivity), and each fiber is an N-torsor (free and transitive kernel action: `kernelAct_free`, `kernelAct_transitive`).
- **2-cocycle and splitting criterion.** `sectionCocycle S` extracts the N-valued cocycle from any set-theoretic section. `splits_iff_trivial_cocycle`: the extension splits if and only if there exists a section whose cocycle is identically 1. This fills a gap explicitly listed as a TODO in Mathlib's `GroupExtension/Defs.lean`.
- **Obstruction theorem.** `not_splits_of_all_cocycles_nontrivial`: if every section has a nontrivial cocycle, the extension does not split.
- **Restriction to subgroups.** `restrictExtension S H` pulls back the extension along a subgroup $H \leq G$. `global_split_implies_local_split`: if the extension splits globally, it splits over every subgroup. The contrapositive (`local_nonsplit_obstructs_global`) gives a local obstruction to global splitting.
- **Direct product example.** `directProductExtension` is the trivially split extension $1 \to N \to N \times G \to G \to 1$ with identically trivial cocycle.
- **Section difference and independence.** `sectionDiff` extracts the N-valued difference between two sections (both land in the same fiber). `splitting_independent_of_section`: the splitting question depends only on the existence of *some* section with trivial cocycle, not on a specific choice.

20.1 Faithfully flat descent

A second application domain: for a faithfully flat algebra map $R \to S$, base change reflects injectivity, surjectivity, and bijectivity of ring homomorphisms. In the architecture's language, the forgetful map (base change) is **conservative**—it reflects all three fundamental properties, not merely surjectivity. This is a stronger form of the pattern than group extensions, where the forgetful map is surjective but does not reflect the splitting property.

We formalize this in `GeneralMethod/Descent/FaithfullyFlatDescent.lean`, packaging Mathlib's descent theorems (`codescendsAlong_injective`, etc.) as architecture statements. The two application domains together demonstrate the architecture's range: group extensions exhibit a *non-conservative* forgetful map (set-theoretic section always exists, algebraic section may not), while faithfully flat descent exhibits a *conservative* one (bare properties fully determine enriched properties).

Since the initial development of the group extension material in this paper, companion work [7, 8] has carried the architecture further: `splits_iff_trivial_cocycle` is now complete with zero `sorry`; abstract embedding problems have been formalized as extension splitting with cocycle obstructions; and Quillen's Theorem A for Galois connections has been proved using the same fiber/section/obstruction pattern. These results confirm that the architecture is not merely organizational but mathematically load-bearing.

21 Limitations and future work

The retrospective tranches do not exhaust all closure or fibration patterns, and the group extension and descent work are first applications rather than a comprehensive treatment. The principal value of the tranche survey is transferability and interface validation; theorem-level mathematical leverage is developed in the companion general-method papers [7, 8], and the unified editorial closure of the program is [9, 10]. Natural extensions include: the full $H^2 \leftrightarrow$ extensions bijection, obstruction-theoretic problems in algebraic topology, and effective descent for module categories.

A Theorem indices (Lean names by module)

`QuotientFiberBenchmark.lean`

Lean name	One-line reading
`BareQuotient`	quotient type as bare certificate layer after collapse.
`EnrichedCarrier`	carrier before quotienting (enriched layer).
`forgetToQuotient`	projection `Quotient.mk` from carrier to quotient.
`QuotientFiber`	fiber over a bare quotient point (subtype of the carrier).
`quotient_fiber_nonempty`	each bare point has a nonempty fiber (from `exists_rep`).
`forgetToQuotient_surjective`	`forgetToQuotient` is surjective onto the quotient.
`forgetToQuotient_hasRightInverse`	projection has a global right inverse (section exists).
`quotientOut_rightInverse_forgetToQuotient`	`Quotient.out` is a right inverse of `forgetToQuotient`.
`canonicalQuotientFiberWitness`	canonical distinguished point in each `QuotientFiber`.
`quotient_fiber_nonempty'`	nonemptiness via `canonicalQuotientFiberWitness`.

`ProductProjectionFiberBenchmark.lean`

Lean name	One-line reading
`forgetFirst`	first projection $X \times Y \to X$.
`ProductFiberAt X Y x`	fiber of `forgetFirst` over `x`.
`productSection`	section $x \mapsto (x, y_0)$ at fixed `y0`.
`product_fiber_nonempty`	nonempty `ProductFiberAt` given `y0`.
`forgetFirst_surjective`	`forgetFirst` surjective given `y0`.
`forgetFirst_hasRightInverse`	`HasRightInverse` for `forgetFirst`.

(continued)

Lean name	One-line reading
`productSection_rightInverse_forgetFirst`	`RightInverse` law.
`canonicalProductFiberWitness`	canonical witness in each fiber.
`product_fiber_nonempty'`	nonemptiness via canonical witness.

SigmaFiberBenchmark.lean

Lean name	One-line reading
`SigmaFiber`	fiber of `Sigma.fst` over `b`.
`sigma_fiber_nonempty`	nonempty fibers from global nonemptiness.
`sigma_fst_surjective`	`Sigma.fst` surjective.
`sigma_fst_hasRightInverse`	`HasRightInverse`.
`chooseFiber`	`Classical.choice` in each fiber.
`sigmaSection`	global section of the bundle.
`sigmaSection_rightInverse_sigma_fst`	`RightInverse` law.
`canonicalSigmaFiberWitness`	canonical witness in each `SigmaFiber`.

SumCoproductFiberBenchmark.lean

Lean name	One-line reading
`forgetSumIsLeft`	discriminant collapse `Sum A B → Bool`.
`SumFiberAt`	fiber over a side tag.
`sumSection`	parametric section from `Bool`.
`sum_fiber_nonempty`	nonempty fibers (given `a0`, `b0`).
`forgetSumIsLeft_surjective`	surjectivity.
`forgetSumIsLeft_hasRightInverse`	`HasRightInverse`.
`sumSection_rightInverse_forgetSumIsLeft`	`RightInverse` law.
`canonicalSumFiberWitness`	canonical witness in each fiber.

SubtypeValFiberBenchmark.lean

Lean name	One-line reading
`SubtypeFiberAt`	fiber of `Subtype.val` over `a`.
`subtype_fiber_nonempty`	nonempty fibers from $\forall a,\ p\ a$.
`subtype_val_surjective`	surjectivity of `Subtype.val`.
`subtype_val_hasRightInverse`	`HasRightInverse`.
`subtypeSection`	section sending `a` to `<a, h a>`.
`subtypeSection_rightInverse_subtypeVal`	`RightInverse` law.
`canonicalSubtypeFiberWitness`	canonical witness in each fiber.

OrbitRelationQuotientFiberBenchmark.lean

Lean name	One-line reading
`same_orbit`	orbit relation from a `MulAction`.
`grp_action_orbit_setoid`	`Setoid` for orbit equivalence.

(continued)

Lean name	One-line reading
`BareOrbitQuotient`	orbit-space quotient type.
`to_orbit_quotient`	collapse map `Quotient.mk`.
`OrbitQuotientFiber`	fiber over an orbit point.
`orbit_quotient_fiber_nonempty`	nonempty fibers.
`to_orbit_quotient_surjective`	surjectivity.
`to_orbit_quotient_hasRightInverse`	`HasRightInverse`.
`quotient_out_rightInverse_to_orbit_quotient`	`Quotient.out` as `RightInverse`.
`canonical_orbit_quotient_fiber_witness`	canonical witness in each fiber.

IdealQuotientFiberBenchmark.lean

Lean name	One-line reading
`forgetToIdealQuotient`	`Ideal.Quotient.mk I`.
`IdealQuotientFiber`	fiber over a coset in R/I.
`ideal_quotient_fiber_nonempty`	nonempty fibers.
`forgetToIdealQuotient_surjective`	surjectivity.
`forgetToIdealQuotient_hasRightInverse`	`HasRightInverse`.
`idealQuotientOut_rightInverse_forgetToIdealQuotient`	`Quotient.out` as `RightInverse`.
`canonicalIdealQuotientFiberWitness`	canonical witness in each fiber.

PushoutCoequalizerFiberBenchmark.lean

Lean name	One-line reading
`pushoutRel`	generating symmetric edges for glueing.
`pushoutSetoid`	`EqvGen.setoid` presentation.
`forgetToPushout`	collapse to the pushout quotient.
`PushoutFiber`	fiber over a glued class.
`forgetToPushout_surjective`	surjectivity.
`quotientOut_rightInverse_forgetToPushout`	`Quotient.out` as `RightInverse`.
`canonicalPushoutFiberWitness`	canonical witness in each fiber.

SetClassifierFiberBenchmark.lean

Lean name	One-line reading
`membershipClassifier`	characteristic map `x in S` to `Prop`.
`ClassifierFiber`	fiber of `Subtype.val` over `x`.
`classifier_fiber_nonempty_of_mem`	nonempty when `x` lies in `S`.
`subtypeVal_surjectiveOn`	surjectivity under global membership.
`subtypeVal_hasRightInverse`	`HasRightInverse` under global membership.
`classifierSection_rightInverse_subtypeVal`	`RightInverse` law.

LocalizationPairFiberBenchmark.lean

Lean name	One-line reading
forgetPairToLocalization	IsLocalization.mk' on R × M.
forgetPairToLocalization_surjective	surjectivity of the pair map.
forgetPairToLocalization_hasRightInverse	HasRightInverse.
sec_rightInverse_forgetPairToLocalization	IsLocalization.sec as RightInverse.
LocalizationPairFiber	fiber over a localized element.
canonicalLocalizationPairFiberWitness	canonical pair witness.

PullbackFiberBenchmark.lean

Lean name	One-line reading
PullbackType	pullback as subtype of A × B.
pullbackFst	first projection (renamed; avoids clash with T2).
pullbackFst_surjective	surjectivity under overlap hypothesis.
pullbackFst_hasRightInverse	HasRightInverse.
pullbackSection_rightInverse_pullbackFst	RightInverse for chosen section.

NonSurjectiveSuccFiberBenchmark.lean

Lean name	One-line reading
succFiber	fiber of Nat.succ.
succFiber_nonempty_iff	nonempty iff not zero.
nat_succ_not_surjective	no global section onto Nat.
nat_succ_not_hasRightInverse	obstruction to HasRightInverse.
pred_leftInverse_succ	Nat.pred as left inverse of succ.

FiberArchitecture.lean (group extensions)

Lean name	One-line reading
ExtensionFiber	preimage fiber of the quotient map.
extension_fiber_nonempty	nonempty fibers (surjectivity).
rightHom_surjective_hasRightInverse	set-theoretic HasRightInverse.
canonicalFiberWitness	canonical element in each fiber.
kernelActOnFiber	kernel action on fibers.
kernelAct_free	kernel acts freely.
kernelAct_transitive	kernel acts transitively (N-torsor).
splitting_iff_section_is_homomorphism	splitting iff some section is a homomorphism.

SchurZassenhaus.lean (2-cocycle and splitting)

Lean name	One-line reading
sectionCocycle	N-valued 2-cocycle of a set-theoretic section.

(continued)

Lean name	One-line reading
`sectionCocycle_spec`	defining equation for the cocycle.
`section_mul_eq`	decomposition: $\sigma(g_1)\sigma(g_2) = \iota(c(g_1, g_2))\,\sigma(g_1 g_2)$.
`splits_iff_trivial_cocycle`	splits iff some section has trivial cocycle.

`LocalGlobal.lean` **(restriction and examples)**

Lean name	One-line reading
`directProductExtension`	trivially split extension $N \times G$.
`directProductSplitting`	canonical splitting of the direct product.
`directProduct_has_trivial_cocycle`	trivial cocycle for the splitting section.
`not_splits_of_all_cocycles_nontrivial`	obstruction: nontrivial cocycles block splitting.
`pullbackSubgroup`	preimage subgroup $\pi^{-1}(H)$.
`restrictExtension`	pullback extension to a subgroup $H \leq G$.
`splitting_restricts`	global splitting restricts to subgroups.
`global_split_implies_local_split`	global split implies local split.
`local_nonsplit_obstructs_global`	contrapositive: local obstruction blocks global.

B A workflow for reusing this pattern

The benchmarks above do *not* constitute a single new lemma or tactic that resolves goals that were previously out of reach. They exhibit a **reusable organization**: choose a surjection or collapse map $\pi : E \to B$, describe the fibers, state a minimal existence theorem as baseline, then—when it fits—strengthen the interface with surjectivity of π, existence of a (possibly classical) section, and canonical choices in fibers (Section 4). This appendix gives a step-by-step way to apply that organization to another problem, including problems that remain partly or fully open.

Scope. The workflow clarifies *where* a forgetful map and fiber structure appear, and whether a global right inverse is even available. It does not replace domain mathematics: restating a conjecture in fiber language rarely proves it. New ideas still come from the field; this pattern helps one see which lemmas would sit at the boundary between a coarse certificate and the data that varies above it.

Step 1: isolate the baseline theorem. Write the statement a working mathematician would accept as the minimal "obvious" layer: existence of representatives, existence of preimages under a surjection, or other standard representability. That statement is the floor; anything layered on top should imply it, not replace it.

Step 2: identify B, E, and π. Fix the set or type B of coarse certificates, a type E of richer structure, and a map $\pi : E \to B$ whose fibers describe what is not determined by the certificate alone. Use Table 13 and Section 17 as a menu: quotients and glueing, products and dependent sums, tags and subtypes, group or ideal quotients, localization via pairs, pullbacks, or—when π is not onto—the phenomenon illustrated by successor on the natural numbers (nonempty fibers only on the image, partial inverses instead of a global section).

Step 3: formalize the baseline in Lean. Prove or import the minimal facts in Mathlib style. If the aim is to check portability of the pattern *without* leaning on a large custom library, keep the formalization in a small standalone module that depends only on Mathlib (as in the benchmark files accompanying this paper).

Step 4: strengthen when possible. When π is surjective, it is often natural to record: nonempty fibers, surjectivity of π, existence of a right inverse in the sense of `HasRightInverse`, a concrete `RightInverse`, and a canonical element of each fiber (Section 4). When some of this fails, the failure is itself information: for example, there may be no global section, only restriction to a subdomain or a one-sided inverse (as with predecessor and successor). State that boundary case explicitly rather than forcing a full section bundle.

Step 5: open problems and partial progress. If the main theorem is still open, use the same decomposition to list separate conjectures: what belongs to the certificate layer, what belongs to fiber data, what would be a section or choice principle. Partial results can attach cleanly to one of these layers. Progress on a hard problem may then consist of proving a lemma at that interface—even when the headline claim remains open—rather than of relabeling the conjecture.

References

[1] N. Spivack. *Canonical Certification Does Not Exhaust Reflective Structure: Reflective Split, Strict Refinement, and Fiber Structure in Infinity Compression*. Flagship manuscript, 2026. Companion LaTeX source: `papers/Canonical_Certification_and_Enriched_Reflective_Realization/`.

[2] N. Spivack. *Strata: Abstract Synthesis and Bridge Laws (Infinity Compression)*. Companion manuscript in preparation, 2026.

[3] Infinity Compression contributors. *EPIC_015_WV1 — External Proof Architecture Validation* (specification and validation verdict notes). `infinity-compression` repository, `specs/` tree, 2026.

[4] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction — CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[5] The Mathlib community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pages 367–381, 2020. DOI: 10.1145/3372885.3373824. Mathlib 4 repository: https://github.com/leanprover-community/mathlib4.

[6] S. Mac Lane. *Categories for the Working Mathematician*, chapter I (categories of algebraic structures; universal constructions motivating quotients). Springer, 2nd edition, 1998.

[7] N. Spivack. *Fiber Architecture for Group Extensions in Lean 4: Cocycles, Splitting, and the Cohomological Bridge*. General-method manuscript, 2026.

[8] N. Spivack. *Quillen's Theorem A for Galois Connections: A Machine-Checked Formalization in Lean 4*. General-method manuscript, 2026.

[9] N. Spivack. *Certification, Realization, and Obstruction: A Universal Fiber Architecture*. Synthesis manuscript, 2026.

[10] N. Spivack. *Reflective Non-Exhaustion: Certification, Realization, Fibers, and Obstruction*. Summit manuscript, 2026.